



Lecture 34 (Sorting 3)

Quicksort

CS61B, Spring 2026 @ UC Berkeley

Josh Hug and Kay Ousterhout

Sorts So Far

	Best Case Runtime	Worst Case Runtime	Space	Notes
Selection Sort	$\Theta(N^2)$ Find smallest item in remaining items, N times	$\Theta(N^2)$	$\Theta(1)$	
Heapsort (in place)	$\Theta(N)$ All duplicates	$\Theta(N \log N)$	$\Theta(1)^{**}$	Bad cache (61C) performance.
Mergesort	$\Theta(N \log N)$	$\Theta(N \log N)$	$\Theta(N)$	Faster than heapsort.
Insertion Sort (in place)	$\Theta(N)$ cN Inversions	$\Theta(N^2)$	$\Theta(1)$	Best for small N or almost sorted.

See [this link](#) for bonus slides on Shell's Sort, an optimization of insertion sort.

Quicksort

Lecture 34, CS61B, Spring 2026

Quicksort

Quicksort Partitioning

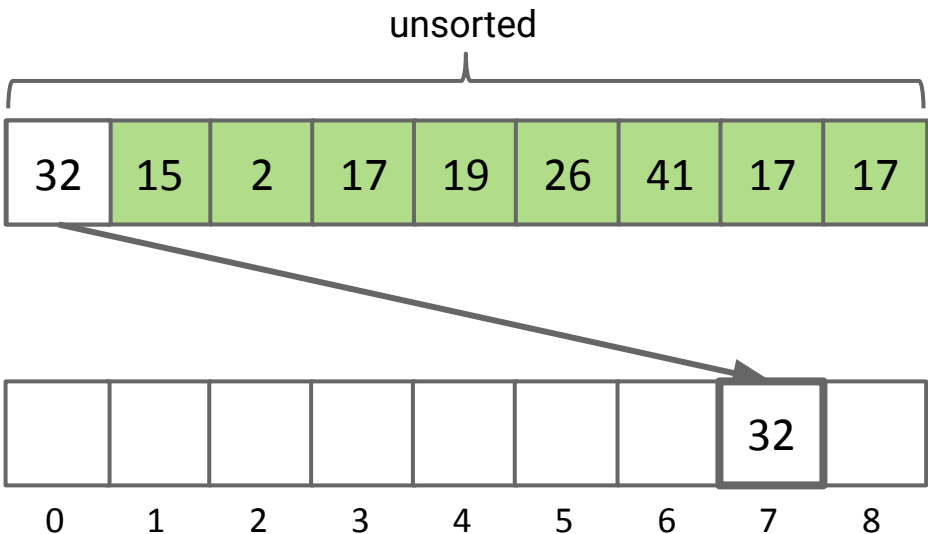
- Naive Partitioning
- Hoare Partitioning

Quicksort Runtime Analysis

- Quicksort Runtime
- Philosophies for Avoiding Worst Case Behavior

Quick Select (median finding) (extra)

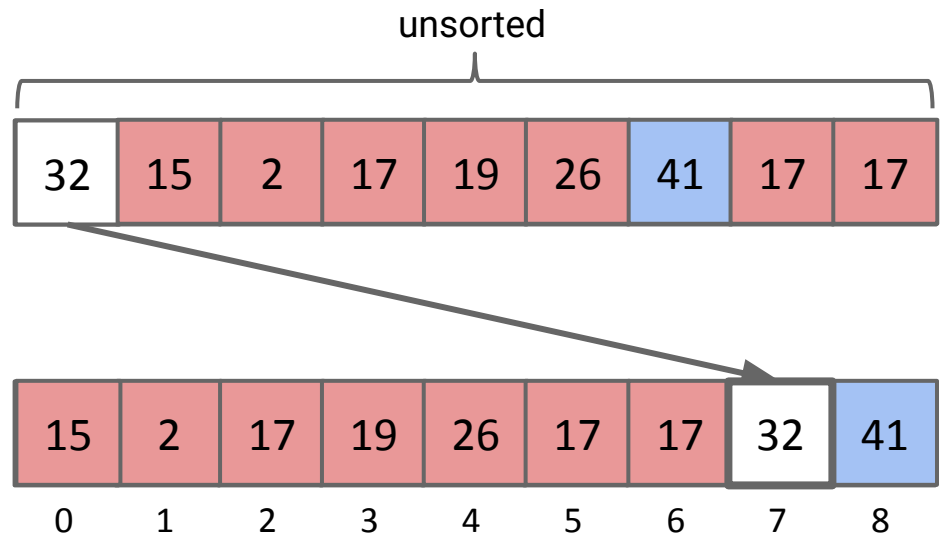
Input:



Pick one item (“pivot”),
get it in the correct spot

Do this to each item => sorted array

Input:

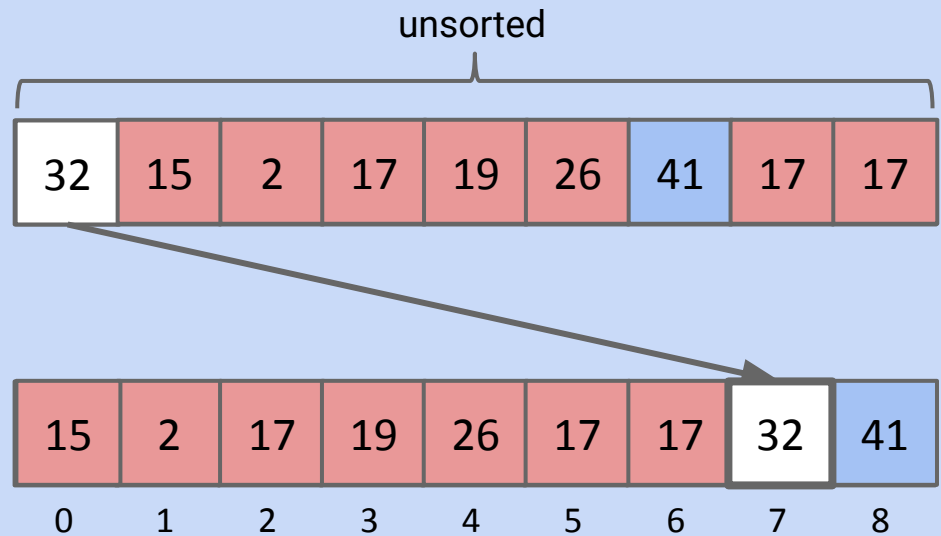


Pick one item ("pivot"),
get it in the correct spot

Get everything else on the
correct side

Do this to each item => sorted array

Input:



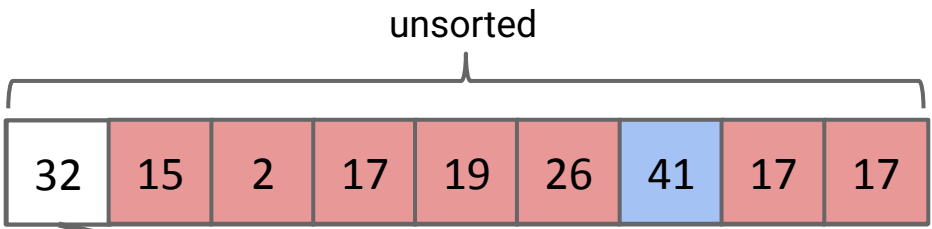
Pick one item ("pivot"),
get it in the correct spot

Get everything else on the
correct side

Why?

Do this to each item => sorted array

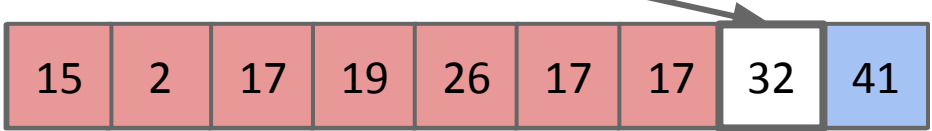
Input:



Partition

Pick one item ("pivot"),
get it in the correct spot

Get everything else on the
correct side



Quicksort

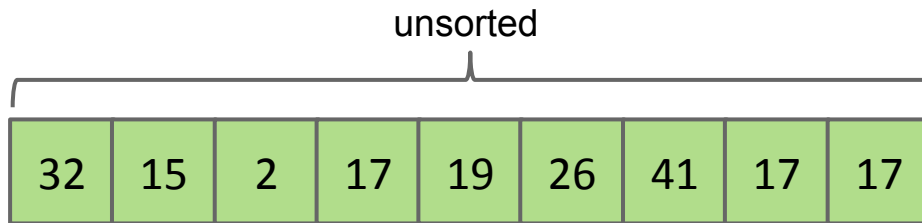
Quicksort

**Gives us two smaller problems to
solve!**

Quick sorting N items:

- Partition on leftmost item.
- Quicksort left half.
- Quicksort right half.

Input:



Quick sorting N items:

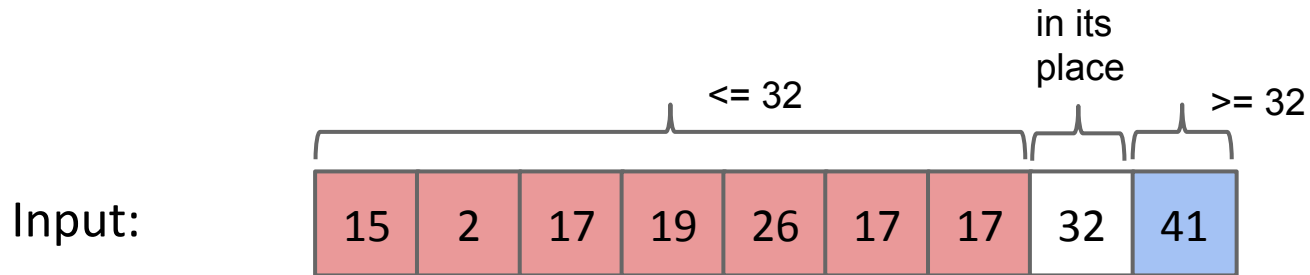
- **Partition on leftmost item (32).**
- Quicksort left half.
- Quicksort right half.

Input:

32	15	2	17	19	26	41	17	17
----	----	---	----	----	----	----	----	----

Quick sorting N items:

- **Partition on leftmost item (32).**
- Quicksort left half.
- Quicksort right half.



Quick sorting N items:

- **Partition on leftmost item (32) (done).**
- Quicksort left half.
- Quicksort right half.

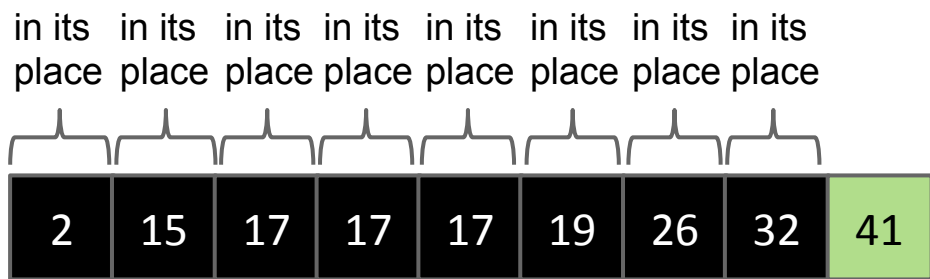
Input:



Quick sorting N items:

- Partition on leftmost item (32) (done).
- **Quicksort left half (details not shown).**
- Quicksort right half.

Input:

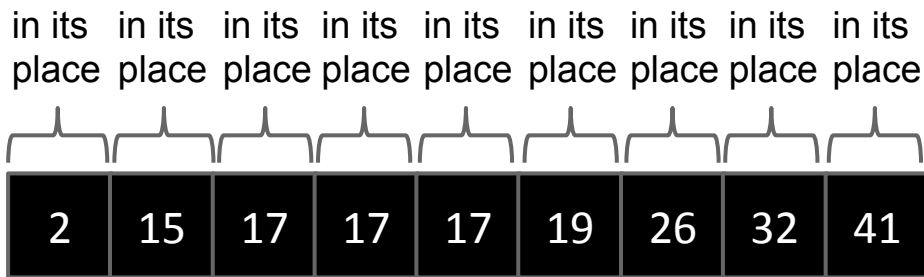


Quick sorting N items:

- Partition on leftmost item (32) (done).
- Quicksort left half (details not shown).
- **Quicksort right half (details not shown).**

If you don't fully trust
the recursion, see
[these extra slides](#) for
a complete demo.

Input:



Naive Partitioning

Lecture 34, CS61B, Spring 2026

Quicksort

Quicksort Partitioning

- **Naive Partitioning**
- Hoare Partitioning

Quicksort Runtime Analysis

- Quicksort Runtime
- Philosophies for Avoiding Worst Case Behavior

Quick Select (median finding) (extra)

A **partition** of an array, given a **pivot** x , is a rearrangement of the items so that:

- All entries to the left of x are $\leq x$.
- All entries to the right of x are $\geq x$.



Called the 'pivot'.

5	550	10	4	10	9	330
---	-----	----	---	----	---	-----

A.

4	5	9	10	10	330	550
---	---	---	----	----	-----	-----

B.

5	4	9	10	10	550	330
---	---	---	----	----	-----	-----

C.

5	9	10	4	10	550	330
---	---	----	---	----	-----	-----

D.

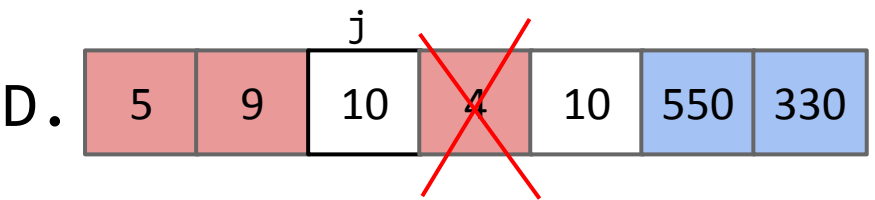
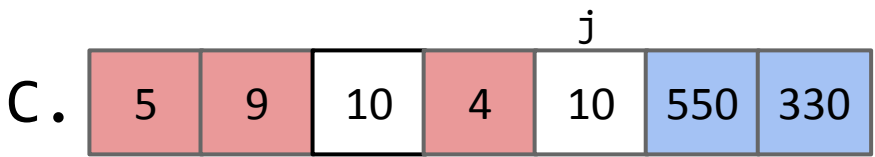
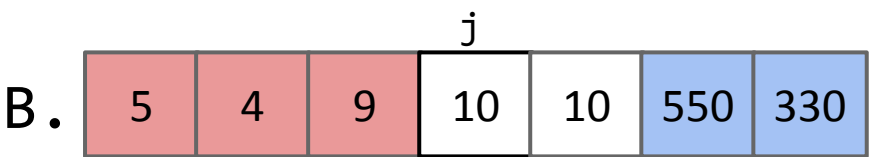
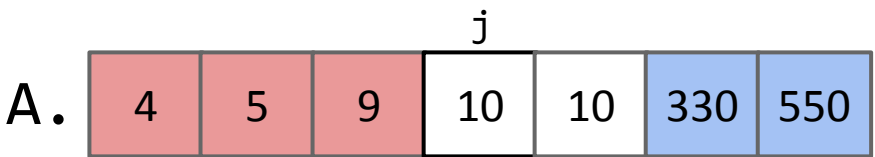
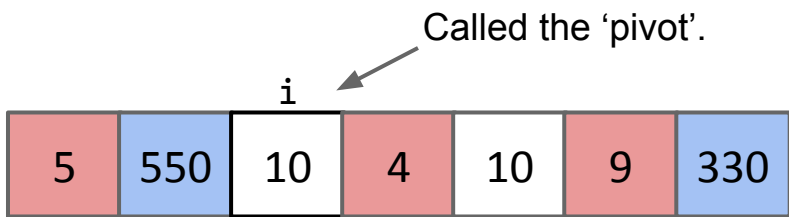
5	9	10	4	10	550	330
---	---	----	---	----	-----	-----

Which partitions are valid?

The Core Idea of Tony's Sort: Partitioning

A **partition** of an array, given a **pivot** x , is a rearrangement of the items so that:

- All entries to the left of x are $\leq x$.
- All entries to the right of x are $\geq x$.



Which partitions are valid?



Job Interview Style Question (Partitioning)

Given an array of colors where the 0th element is white, and the remaining elements are red (less) or blue (greater), rearrange the array so that all red squares are to the left of the white square, and all blue squares are to the right. Your algorithm must complete in $\Theta(N)$ time (no space restriction).

- Relative order of red and blues does NOT need to stay the same.

Input



Example of a valid output



Another example of a valid output



Quicksort Partitioning, take 1

Algorithm: Create another array. Scan and copy all the red items to the first R spaces. Then scan for and copy the white item. Then scan and copy the blue items to the last B spaces.

Input

6	8	3	1	2	7	4
---	---	---	---	---	---	---

Output

3	1	2	4	6	8	7
---	---	---	---	---	---	---

Student asks: "Is it possible to do in one scan?" Yes! Some of your ideas did this.
Student observed: We can do this in constant space (coming soon).

Hoare Partitioning

Lecture 34, CS61B, Spring 2026

Quicksort

Quicksort Partitioning

- Naive Partitioning
- **Hoare Partitioning**

Quicksort Runtime Analysis

- Quicksort Runtime
- Philosophies for Avoiding Worst Case Behavior

Quick Select (median finding) (extra)

Tony originally proposed a scheme where two pointers walk towards each other.

- Left pointer loves small items.
- Right pointer loves large items.
- Big idea: Walk towards each other, swapping anything they don't like.
 - End result is that things on left are “small” and things on the right are “large”.

(Note: The demo we'll show is not the exact scheme Tony used)

Hoare Partitioning

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- Walk pointers towards each other, stopping on a hated item.
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done.

Input:

17	15	19	32	2	26	41	17	17
----	----	----	----	---	----	----	----	----

L

Get rid of large or
equal items

G

Get rid of small or
equal items

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- **Walk pointers towards each other**, stopping on a hated item.
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done.

Input:

17	15	19	32	2	26	41	17	17
----	----	----	----	---	----	----	----	----

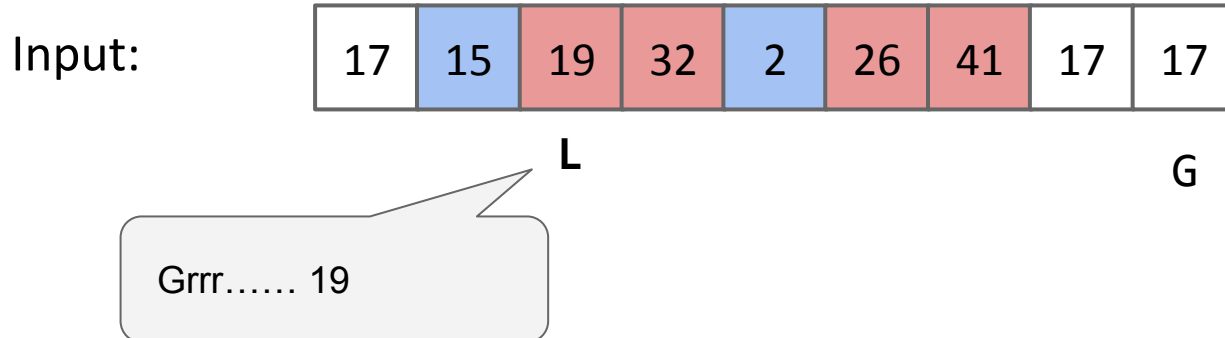
L

G

Hello, lovely 15.

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- **Walk pointers towards each other, stopping on a hated item.**
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done.

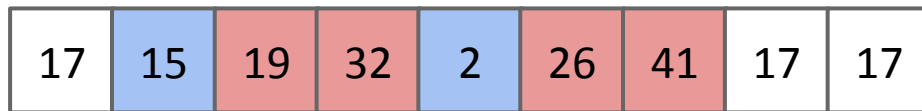


Hoare Partitioning

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- **Walk pointers towards each other, stopping on a hated item.**
 - When both pointers have stopped, swap and move pointers by one.

Input:



L

G

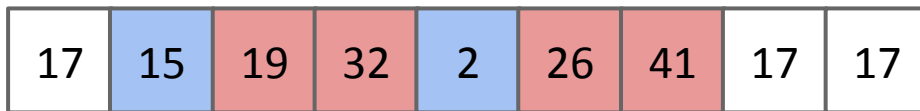
I dislike 17.

Hoare Partitioning

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- Walk pointers towards each other, stopping on a hated item.
 - **When both pointers have stopped, swap and move pointers by one.**
- When pointers cross, you are done.

Input:



L

Time to swap.

G

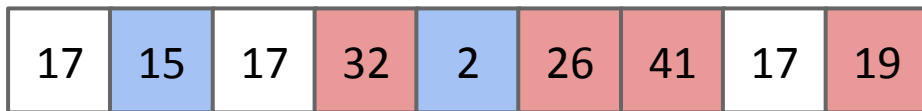
Time to swap.

Hoare Partitioning

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- Walk pointers towards each other, stopping on a hated item.
 - **When both pointers have stopped, swap and move pointers by one.**
- When pointers cross, you are done.

Input:



L

Swapped.

G

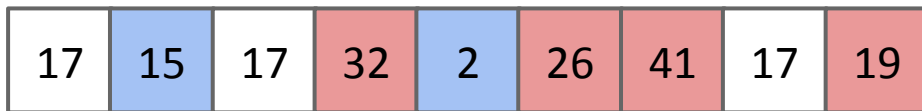
Swapped.

Hoare Partitioning

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- **Walk pointers towards each other, stopping on a hated item.**
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done.

Input:



L

G

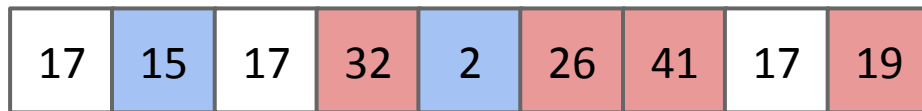
Immediate trouble.

Hoare Partitioning

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- **Walk pointers towards each other, stopping on a hated item.**
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done.

Input:



L

G

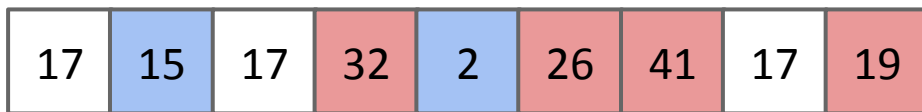
Trouble here, too.

Hoare Partitioning

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- Walk pointers towards each other, stopping on a hated item.
 - **When both pointers have stopped, swap and move pointers by one.**
- When pointers cross, you are done walking.

Input:



L

G

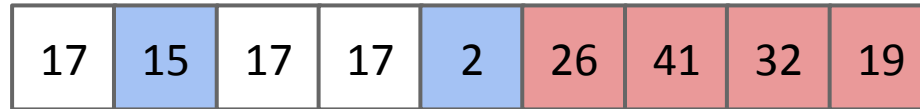
Time to swap.

Time to swap.

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- Walk pointers towards each other, stopping on a hated item.
 - **When both pointers have stopped, swap and move pointers by one.**
- When pointers cross, you are done walking.

Input:



L

G

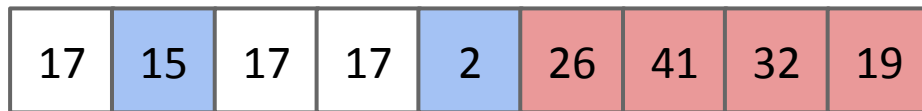
Swapped!

Swapped!

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- **Walk pointers towards each other, stopping on a hated item.**
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done walking.

Input:



L

G

2 is cool.

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- **Walk pointers towards each other, stopping on a hated item.**
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done walking.

Input:

17	15	17	17	2	26	41	32	19
----	----	----	----	---	----	----	----	----

L G

26 is grossly large.

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- **Walk pointers towards each other, stopping on a hated item.**
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done walking.

Input:

17	15	17	17	2	26	41	32	19
----	----	----	----	---	----	----	----	----

L

G

41 is fine.

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- **Walk pointers towards each other, stopping on a hated item.**
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done walking.

Input:

17	15	17	17	2	26	41	32	19
----	----	----	----	---	----	----	----	----

L

G

26 is fine.

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- **Walk pointers towards each other, stopping on a hated item.**
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done walking.

Input:

17	15	17	17	2	26	41	32	19
----	----	----	----	---	----	----	----	----

G **L**

2 is no good...
also hi L!!

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- Walk pointers towards each other, stopping on a hated item.
 - When both pointers have stopped, swap and move pointers by one.
- **When pointers cross, you are done walking.**

Input:

17	15	17	17	2	26	41	32	19
----	----	----	----	---	----	----	----	----

G L

2 is no good...
also hi L!!

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- Walk pointers towards each other, stopping on a hated item.
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done walking.
- **Swap pivot with G** [this step isn't in Tony Hoare's original scheme, but is equivalent].

Input:

17	15	17	17	2	26	41	32	19
----	----	----	----	---	----	----	----	----

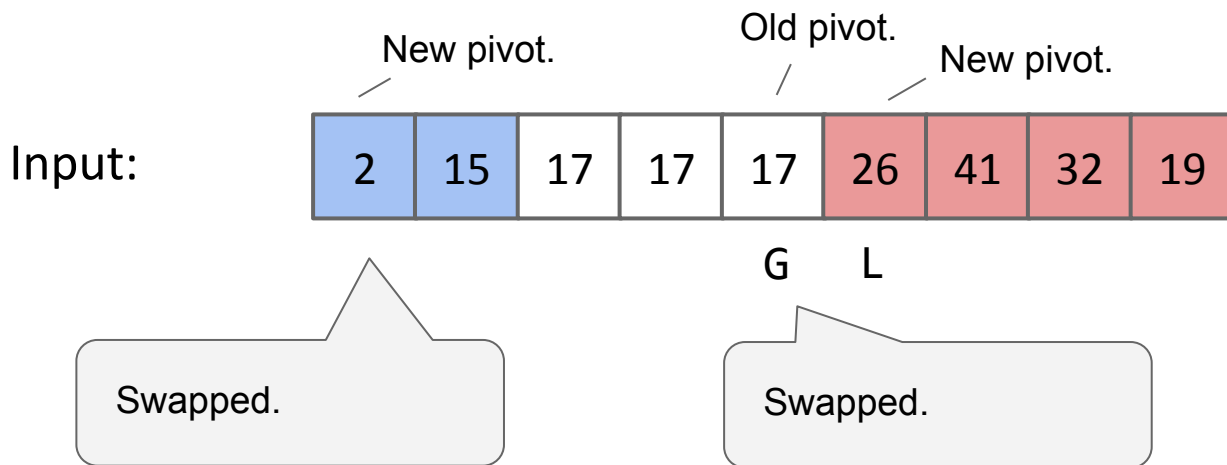
G L

Time to swap.

Time to swap.

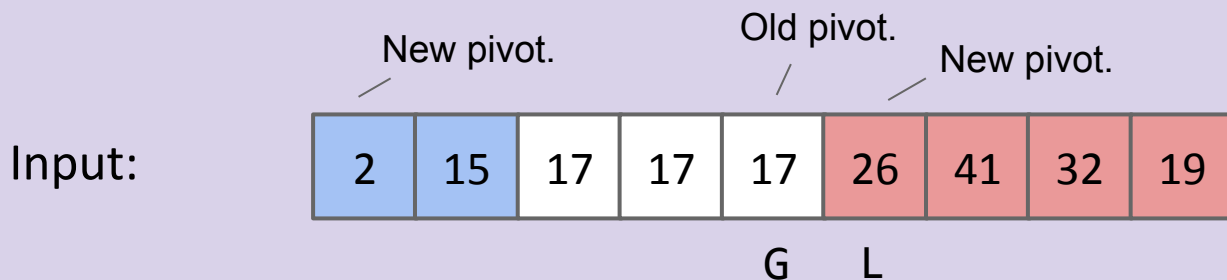
Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- Walk pointers towards each other, stopping on a hated item.
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done walking.
- **Swap pivot with G** [this step isn't in Tony Hoare's original scheme, but is equivalent].



Create L and G pointers at left and right ends.

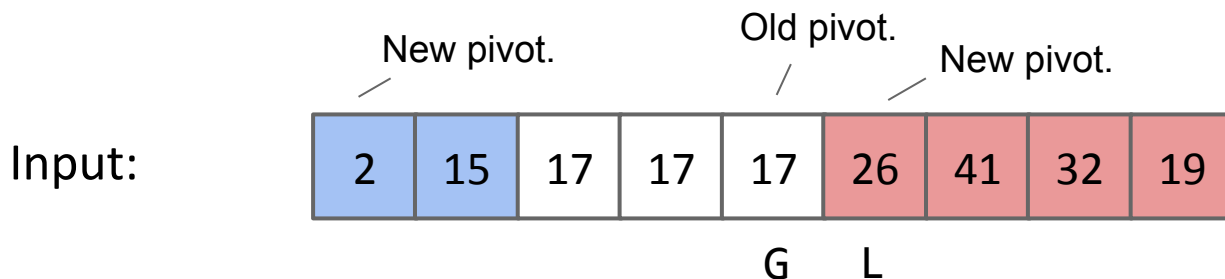
- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- Walk pointers towards each other, stopping on a hated item.
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done walking.
- Swap pivot with G [this step isn't in Tony Hoare's original scheme, but is equivalent].



What's the time and space complexity of Hoare Partitioning?

Create L and G pointers at left and right ends.

- L pointer is a friend to small items, and hates large or equal items.
- G pointer is a friend to large items, and hates small or equal items.
- Walk pointers towards each other, stopping on a hated item.
 - When both pointers have stopped, swap and move pointers by one.
- When pointers cross, you are done walking.
- Swap pivot with G [this step isn't in Tony Hoare's original scheme, but is equivalent].



Time Complexity is $\Theta(N)$; Space Complexity is $\Theta(1)$ (no array copy!)

Quicksort Runtime Analysis

Lecture 34, CS61B, Spring 2026

Quicksort

Quicksort Partitioning

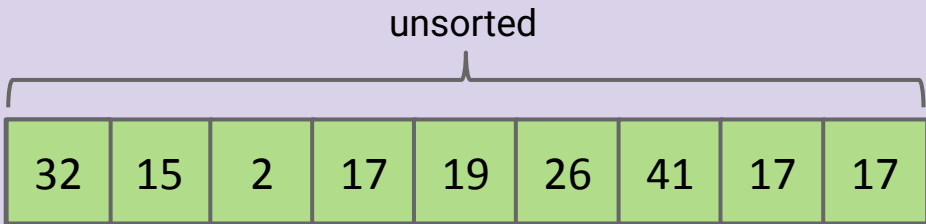
- Naive Partitioning
- Hoare Partitioning

Quicksort Runtime Analysis

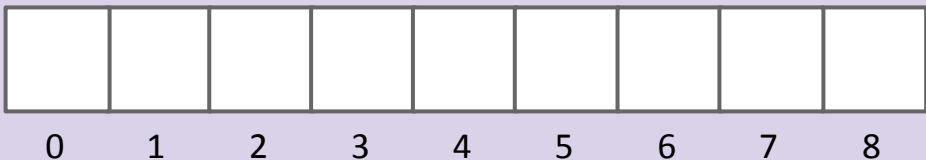
- **Quicksort Runtime**
- Philosophies for Avoiding Worst Case Behavior

Quick Select (median finding) (extra)

Input:



Pick one item (“pivot”),
get it in the correct spot



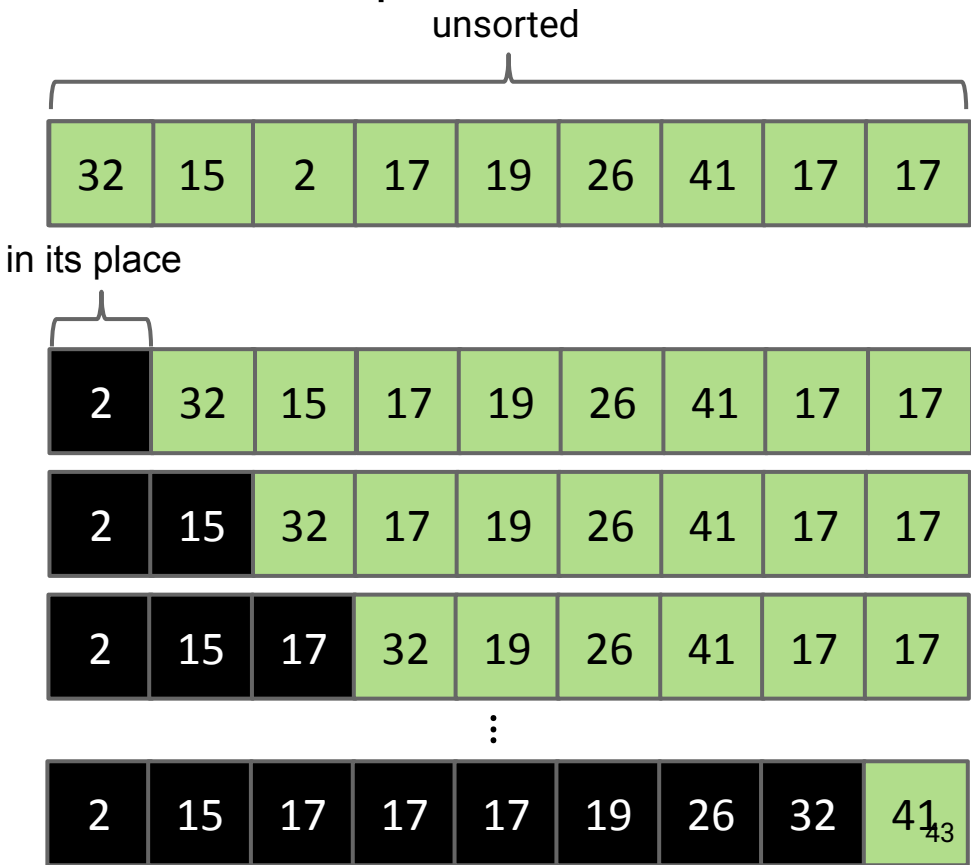
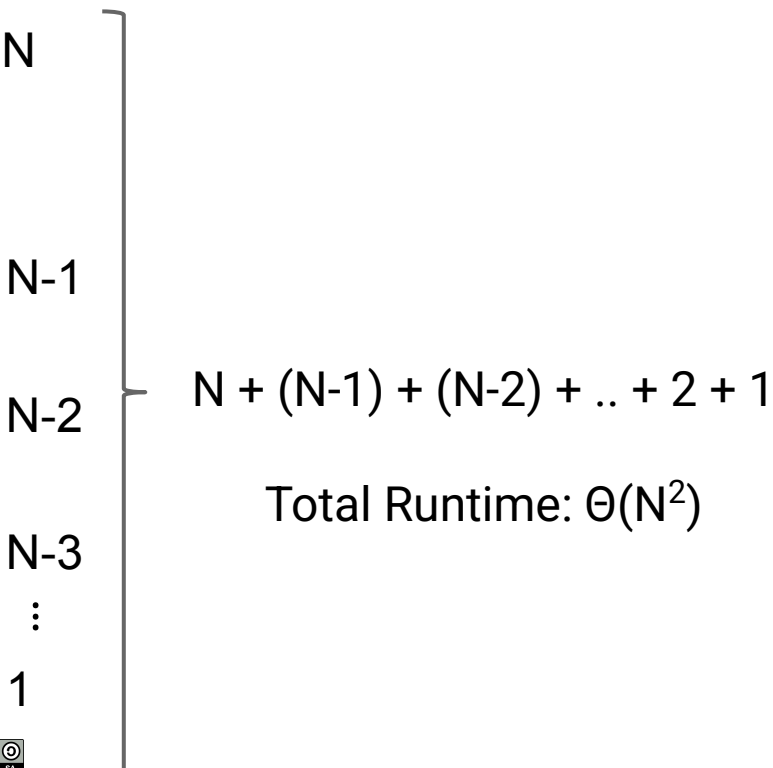
Which pivot should we choose to make runtime as fast as possible?



Choosing a pivot: Smallest Item

Suppose we always choose the smallest item as the pivot.

Work to partition each level:



Worst Case: Pivot Always Lands at Beginning of Array

What is the runtime $\Theta(\cdot)$?

- N^2

Given our pivot scheme (use the first item), give an example of an array that would follow the pattern to the right:

- 1 2 3 4 5 6



Choosing a pivot: Median

Suppose we always choose the median as the pivot

Work to partition each level:

N

$$\approx N/2 + \approx N/2 = \approx N$$

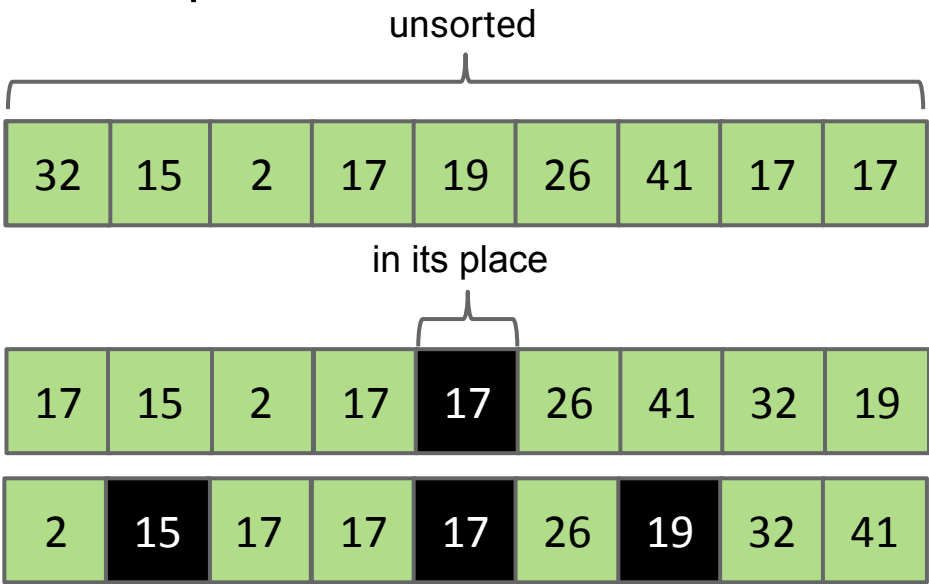
$$\approx N/4 + \approx N/4 + \approx N/4 + \approx N/4 = \approx N$$

⋮

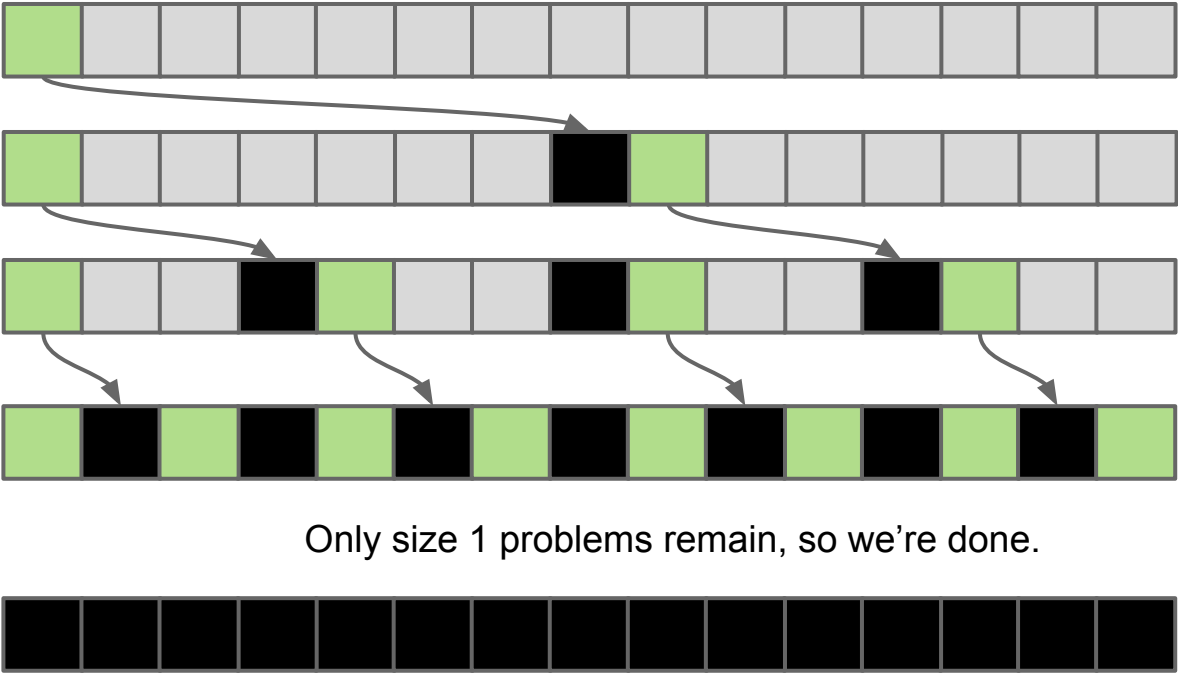
}

Work at each level: N
Number of levels: log N

Total Runtime: $\Theta(N \log N)$



Best Case Runtime?



Total work at each level:

$$\approx N$$

$$\approx N/2 + \approx N/2 = \approx N$$

$$\approx N/4 * 4 = \approx N$$

Overall runtime:
 $\Theta(NH)$ where $H = \Theta(\log N)$

so: $\Theta(N \log N)$

Theoretical analysis:

- Best case: $\Theta(N \log N)$
- Worst case: $\Theta(N^2)$

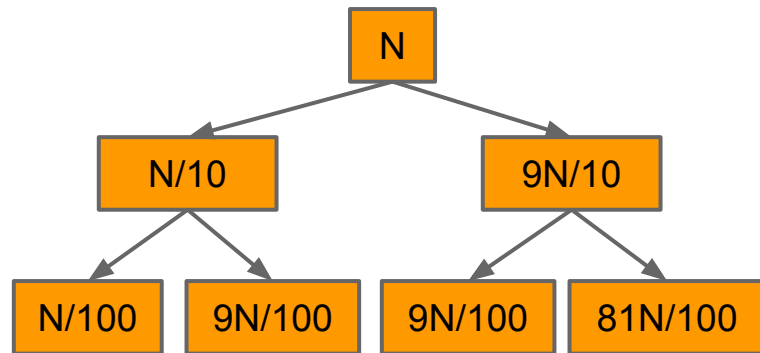
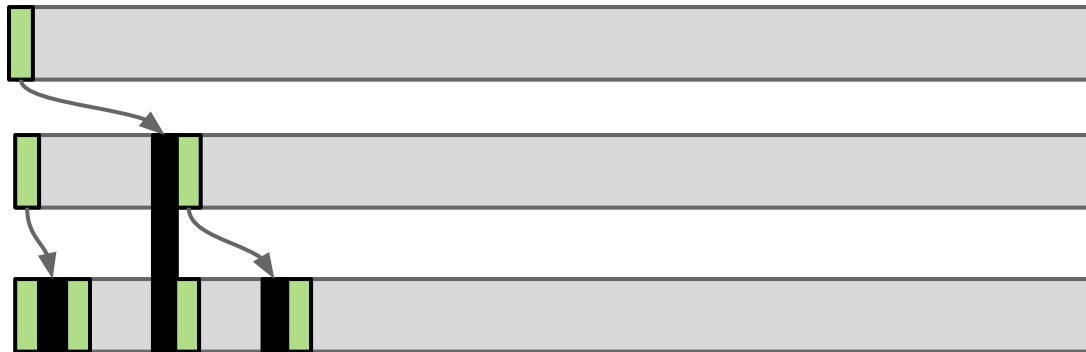
Compare this to Mergesort.

- Best case: $\Theta(N \log N)$
- Worst case: $\Theta(N \log N)$

Recall that $\Theta(N \log N)$ vs. $\Theta(N^2)$ is a **really big deal**. So how can Quicksort be the fastest sort empirically? Because on average it is $\Theta(N \log N)$.

- Rigorous proof requires probability theory + calculus, but intuition + empirical analysis will hopefully convince you.

Suppose pivot always ends up at least 10% from either edge (not to scale).

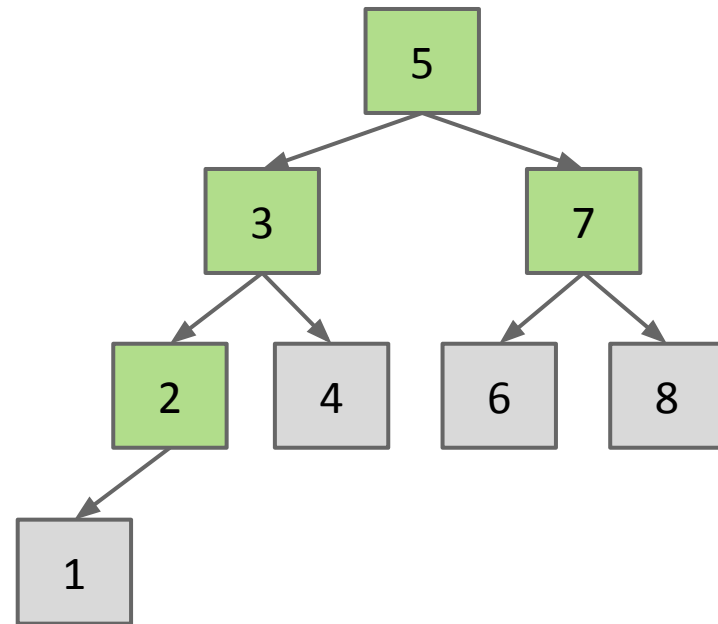


Work at each level: $O(N)$

- Runtime is $O(NH)$.
 - H is approximately $\log_{10/9} N = O(\log N)$
- Overall: $O(N \log N)$.

Punchline: Even if you are unlucky enough to have a pivot that never lands anywhere near the middle, but at least always 10% from the edge, runtime is still $O(N \log N)$.

Argument #2: Quicksort is BST Sort



Key idea: compareTo calls are same for BST insert and Quicksort.

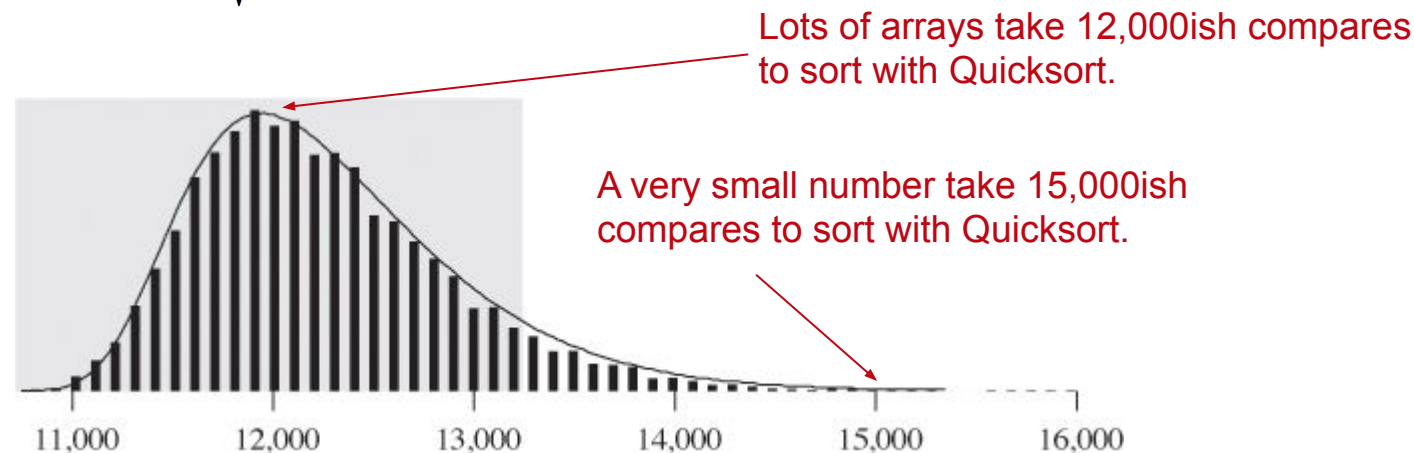
- Every number gets compared to 5 in both.
- 3 gets compared to only 1, 2, 4, and 5 in both.

Reminder: Random insertion into a BST takes $O(N \log N)$ time.

For N items:

- Mean number of compares to complete Quicksort: $\sim 2N \ln N$
- Standard deviation:

$$\sqrt{(21 - 2\pi^2)/3}N \approx 0.6482776N$$



Empirical histogram for quicksort compare counts (10,000 trials with N = 1000)

Chance of taking 1,000,000ish compares is effectively zero.

For more, see: <http://www.informit.com/articles/article.aspx?p=2017754&seqNum=7>

Theoretical analysis:

- Best case: $\Theta(N \log N)$
- Worst case: $\Theta(N^2)$
- **Randomly chosen array case: $\Theta(N \log N)$ expected**

For our pivot/partitioning strategies: Sorted or close to sorted.

With extremely high probability!!

Compare this to Mergesort.

- Best case: $\Theta(N \log N)$
- Worst case: $\Theta(N \log N)$

Why is it faster than mergesort?

- Requires empirical analysis. No obvious reason why.

Sorting Summary (So Far)

	Best Case Runtime	Worst Case Runtime	Space	Notes
Selection Sort	$\Theta(N^2)$ Find smallest item in remaining items, N times	$\Theta(N^2)$	$\Theta(1)$	
Heapsort (in place)	$\Theta(N)$ All duplicates	$\Theta(N \log N)$	$\Theta(1)^{**}$	Bad cache (61C) performance.
Mergesort	$\Theta(N \log N)$	$\Theta(N \log N)$	$\Theta(N)$	Faster than heapsort.
Insertion Sort (in place)	$\Theta(N)$ cN Inversions	$\Theta(N^2)$	$\Theta(1)$	Best for small N or almost sorted.
Quick Sort (in place)	$\Theta(N \log N)$ This is also the runtime on average	$\Theta(N^2)$	$\Theta(\log N)$ call stack	Fastest sort

Listed by mechanism:

- Selection sort: Find the smallest item and put it at the front.
- Insertion sort: Figure out where to insert the current item.
- Merge sort: Merge two sorted halves into one sorted whole.
- Partition (quick) sort: Partition items around a pivot.

Listed by memory and runtime:

	Memory	Time	Notes
Heapsort	$\Theta(1)$	$\Theta(N \log N)$	Bad caching (61C)
Insertion	$\Theta(1)$	$\Theta(N^2)$	$\Theta(N)$ if almost sorted
Mergesort	$\Theta(N)$	$\Theta(N \log N)$	
Quicksort	$\Theta(\log N)$ (call stack)	$\Theta(N \log N)$ expected	Fastest sort

We said Quicksort is the fastest, but this is only true if we make the right decisions about:

- Pivot selection.
- Partition algorithm.
- How we deal with avoiding the worst case (can be covered by the above choices).

We'll call this **Quicksort L3S**

Suppose we run a speed test of Mergesort vs. Quicksort from last time, which had:

- Pivot selection: Always use **leftmost**.
- Partition algorithm: Make an array copy then do **three** scans for red, white, and blue items.
- **Shuffle** before starting (to avoid worst case).

Quicksort vs. Mergesort

	Pivot Selection Strategy	Partition Algorithm	Worst Case Avoidance Strategy	Time to sort 1000 arrays of 10000 ints
Mergesort	N/A	N/A	N/A	1.3 seconds
Quicksort L3S	Leftmost	3-scan	Shuffle	3.2 seconds

Quicksort didn't do so well!

Note: These are unoptimized versions of mergesort and quicksort, i.e. no switching to insertion sort for small arrays.

Quicksort vs. Mergesort

	Pivot Selection Strategy	Partition Algorithm	Worst Case Avoidance Strategy	Time to sort 1000 arrays of 10000 ints
Mergesort	N/A	N/A	N/A	1.3 seconds
Quicksort L3S	Leftmost	3-scan	Shuffle	3.2 seconds
Quicksort LTHS	Leftmost	Tony Hoare	Shuffle	0.9 seconds

Using Tony Hoare's two pointer scheme, Quicksort is better than mergesort!

- More recent pivot/partitioning schemes do somewhat better.
 - Best known Quicksort uses a two-pivot scheme.
 - Interesting note, this version of Quicksort was introduced to the world by a previously unknown guy, in a Java developers forum ([Link](#)).

Note: These are unoptimized versions of mergesort and quicksort, i.e. no switching to insertion sort for small arrays.

Amusingly, Quicksort was the wrong tool for the job. Two issues:

- Language that Tony was using didn't support recursion (so he couldn't easily implement Quicksort).
- Sentences are usually shorter than 15 words (So insertion sort is faster).

Tony Hoare

5/13/13

to jhug 

You are quite right. But I am lucky that I did not realise it at that time.

Remember, machines were then a million times slower than they are now.

Yours,

Tony.

Philosophies for Avoiding Worst Case Behavior

Lecture 34, CS61B, Spring 2026

Quicksort

Quicksort Partitioning

- Naive Partitioning
- Hoare Partitioning

Quicksort Runtime Analysis

- Quicksort Runtime
- **Philosophies for Avoiding Worst Case Behavior**

Quick Select (median finding) (extra)

If pivot always lands somewhere “good”, Quicksort is $\Theta(N \log N)$. However, the very rare $\Theta(N^2)$ cases do happen in practice, e.g.

What specific array inputs yield bad Quicksort runtime?

- 10 9 8 7 6 5 4 3 2 1 (pick 10 first, didn't split up the problem!)
- 1 2 3 4 5 6 7 8 9 10 (same problem, but even worse than reverse order)

Recall, our version of Quicksort has the following properties:

- Leftmost item is always chosen as the pivot.



If pivot always lands somewhere “good”, Quicksort is $\Theta(N \log N)$. However, the very rare $\Theta(N^2)$ cases do happen in practice, e.g.

- Bad ordering: Array already in sorted order (or almost sorted order).

What can we do to avoid worst case behavior?

- Check if it's sorted, and abort! If it's almost sorted, insertion sort!
 - Use the item in the middle as the pivot
- Ask Claude or ChatGPT.
- Shuffle items
- Pick a random index for the pivot
- Find the median and use that as the pivot
 - Find a median-ish: look at a few numbers, pick the median
- Write the first item, pick something larger

Recall,

6	8	3	1	2	7	4
---	---	---	---	---	---	---

 the follow

3	1	2	4	6	8	7
---	---	---	---	---	---	---

Leftmost item is always chosen as the pivot.

What can we do to avoid running into the worst case for QuickSort?

Four philosophies:

1. **Randomness**: Pick a random pivot or shuffle before sorting.
2. **Smarter pivot selection**: Calculate or approximate the median.
3. **Introspection**: Switch to a safer sort if recursion goes too deep.
4. **Preprocess the array**: Could analyze array to see if Quicksort will be slow. No obvious way to do this, though (can't just check if array is sorted, almost sorted arrays are almost slow).

If pivot always lands somewhere “good”, Quicksort is $\Theta(N \log N)$. However, the very rare $\Theta(N^2)$ cases do happen in practice, e.g.

- Bad ordering: Array already in sorted order.

Dealing with bad ordering:

- Strategy #1: Pick pivots randomly.
- Strategy #2: Shuffle before you sort.

The second strategy requires care in partitioning code to avoid $\Theta(N^2)$ behavior on arrays of duplicates.

- Common bug, even in a well known 2010s textbook.



Randomness is necessary for best Quicksort performance! For any pivot selection procedure that is:

- Deterministic
- Constant Time

The resulting Quicksort has a family of dangerous inputs that an adversary could easily generate.

- See McIlroy's "[A Killer Adversary for Quicksort](#)"



Java does this.

Dangerous input

Could calculate the actual median in linear time.

- “Exact median Quicksort” is safe: Worst case $\Theta(N \log N)$, but it is slower than Mergesort.

Raises interesting question though: How do you compute the median of an array?
Extra slides talk about how to do this.

Can also simply watch your recursion depth.

- If it exceeds some critical value (say $10 \ln N$), switch to mergesort.

Java also does this (in addition to approximating the median).

Quick Select (median finding)

Lecture 34 CS61B, Spring 2026

Quicksort

Quicksort Partitioning

- Naive Partitioning
- Hoare Partitioning

Quicksort Runtime Analysis

- Quicksort Runtime
- Philosophies for Avoiding Worst Case Behavior

Quick Select (median finding) (extra)

What If We Don't Want Randomness?

Our approach so far: Use randomness to avoid worst case behavior, but some people don't like having randomness in their sorting routine.

Another approach: Use the median (or an approximation) as our pivot.

Four philosophies:

1. **Randomness:** Pick a random pivot or shuffle before sorting.
2. **Smarter pivot selection:** Calculate or approximate the median.
3. **Introspection:** Switch to a safer sort if recursion goes too deep.
4. **Try to cheat:** If the array is already sorted, don't sort (this doesn't work).

This is what we've been using.



The best possible pivot is the median.

- Splits problem into two problems of size $N/2$.

Obvious approach: Just calculate the actual median and use that as pivot.

- But how?

Goal: Come up with an algorithm for finding the median of an array. Bonus points if your algorithm takes linear time.

Goal: Come up with an algorithm for finding the median of an array. Bonus points if your algorithm takes linear time.

- Sort the array, return the middle item. Takes $\Theta(N \log N)$ time to sort, though.

Computing the exact median would be great for picking an item to partition around. Gives us a “safe quick sort”.

However, it turns out that partitioning can be used to find the exact median.

- The resulting algorithm is the best known median identification algorithm.



Goal, find the median:

9	550	14	6	10	5	330	817	913
---	-----	----	---	----	---	-----	-----	-----

Partition, pivot lands at 2.

6	5	9	550	14	10	330	817	913
---	---	---	-----	----	----	-----	-----	-----

- Not the median. Why?
- So what next? Partition right subproblem, median can't be to the left!

			550	14	10	330	817	913
--	--	--	-----	----	----	-----	-----	-----

Now pivot lands at 6.

			14	10	330	550	817	913
--	--	--	----	----	-----	-----	-----	-----

- Not the median.

Pivot lands at 4. Are we done?

			14	10	330			
--	--	--	----	----	-----	--	--	--

- Yep, $9/2 = 4$.

			10	14	330			
--	--	--	----	----	-----	--	--	--

Worst case performance?

What is the worst case performance for Quick Select? Give an array that causes this worst case (assuming we always pick leftmost item as pivot).

Worst case performance?

What is the worst case performance for Quick Select? Give an array that causes this worst case (assuming we always pick leftmost item as pivot).

Worst asymptotic performance $\Theta(N^2)$ occurs if array is in sorted order.

[1 2 3 4 5 6 7 8 9 10 ... N]

[1 **2 3 4 5 6 7 8 9 10** ... N]

[1 **2 3 4 5 6 7 8 9 10** ... N]

...

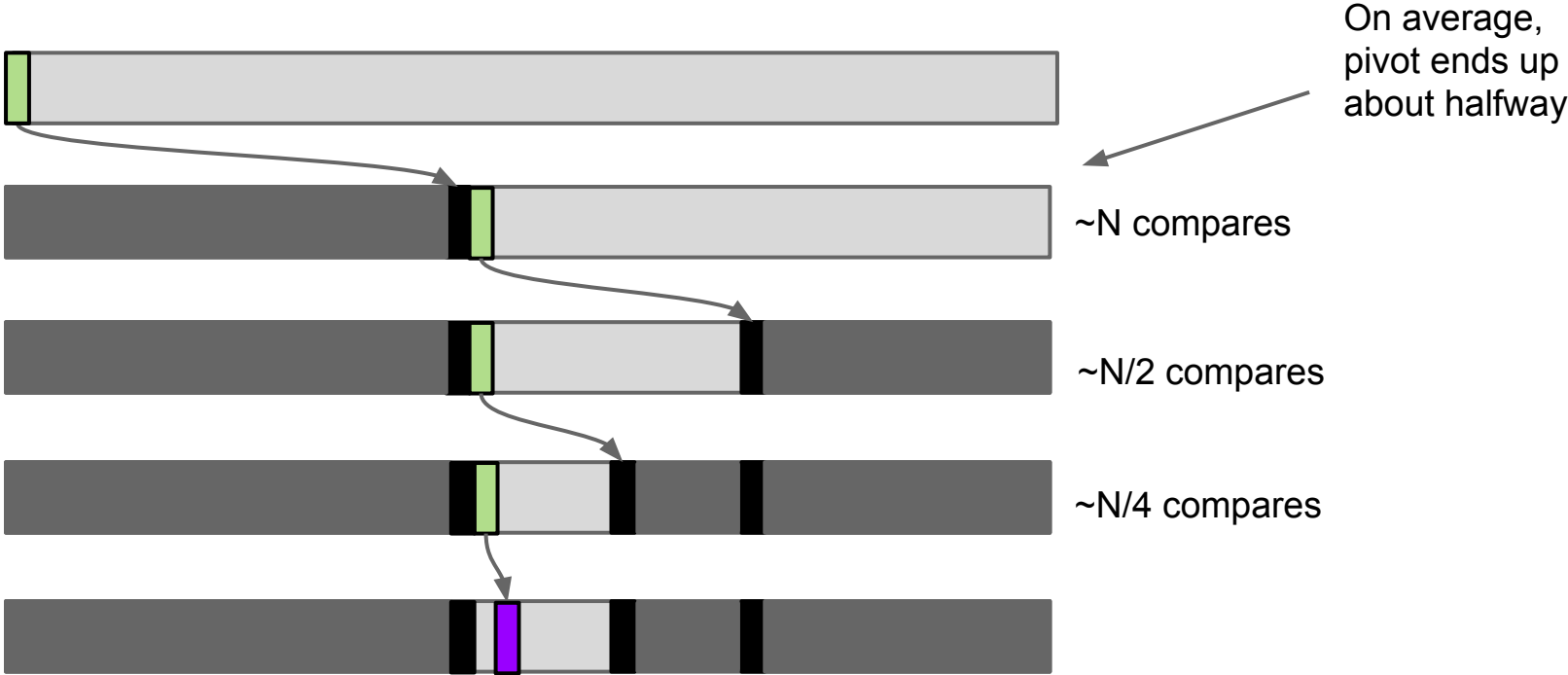
[1 2 3 4 5 ... **N/2** ... N]

Expected Performance

On average, Quick Select will take $\Theta(N)$ time.

- Intuitive picture (not a proof!):

$$N + N/2 + N/4 + \dots + 1 = \Theta(N)$$



Quicksort With Quickselect?

	Pivot Selection Strategy	Partition Algorithm	Worst Case Avoidance Strategy	Time to sort 1000 arrays of 10000 ints	Worst Case
Mergesort	N/A	N/A	N/A	1.3 seconds	$\Theta(N \log N)$
Quicksort L3S	Leftmost	3-scan	Shuffle	3.2 seconds	$\Theta(N^2)$
Quicksort LTHS	Leftmost	Tony Hoare	Shuffle	0.9 seconds	$\Theta(N^2)$
Quicksort QSTH	QuickSelect	Tony Hoare	Exact Median	4.9 seconds	$\Theta(N^2)$

What if we used Quickselect to find the exact median?

- Resulting algorithm is very slow. Doesn't fix the worst case runtime issue
- A little strange to do a bunch of partitions to identify the optimal item to partition around.

Is it possible to find the median in $\Theta(N)$ time?

- Yes! Use '[BFPRT](#)' (called PICK in original paper).
- Algorithm developed in 1972
- In practice, rarely used.

Time Bounds for Selection*

MANUEL BLUM, ROBERT W. FLOYD, VAUGHAN PRATT,
RONALD L. RIVEST, AND ROBERT E. TARJAN

Department of Computer Science, Stanford University, Stanford, California 94305

Received November 14, 1972

Historical note: The authors of this paper include FOUR Turing Award winners (and Vaughan Pratt)

The number of comparisons required to select the i -th smallest of n numbers is shown to be at most a linear function of n by analysis of a new selection algorithm—PICK. Specifically, no more than $5.4305n$ comparisons are ever required. This bound is improved for extreme values of i , and a new lower bound on the requisite number of comparisons is also proved.

Let's see how Exact Median Quicksort performs.

Quicksort vs. Mergesort

	Pivot Selection Strategy	Partition Algorithm	Worst Case Avoidance Strategy	Time to sort 1000 arrays of 10000 ints	Worst Case
Mergesort	N/A	N/A	N/A	1.3 seconds	$\Theta(N \log N)$
Quicksort L3S	Leftmost	3-scan	Shuffle	3.2 seconds	$\Theta(N^2)$
Quicksort LTHS	Leftmost	Tony Hoare	Shuffle	0.9 seconds	$\Theta(N^2)$
Quicksort PickTH	Exact Median	Tony Hoare	Exact Median	~15 seconds	$\Theta(N \log N)$

Quicksort using PICK to find the exact median (Quicksort PickTH) is terrible!

- Cost to compute medians is too high.
- Have to live with worst case $\Theta(N^2)$ if we want good practical performance.

Note: These are unoptimized versions of mergesort and quicksort, i.e. no switching to insertion sort for small arrays.

Listed by mechanism:

- Selection sort: Find the smallest item and put it at the front.
- Insertion sort: Figure out where to insert the current item.
- Merge sort: Merge two sorted halves into one sorted whole.
- Partition (quick) sort: Partition items around a pivot.

Listed by memory and runtime:

	Memory	Time	Notes
Heapsort	$\Theta(1)$	$\Theta(N \log N)$	Bad caching (61C)
Insertion	$\Theta(1)$	$\Theta(N^2)$	$\Theta(N)$ if almost sorted
Mergesort	$\Theta(N)$	$\Theta(N \log N)$	
Random Quicksort	$\Theta(\log N)$ expected	$\Theta(N \log N)$ expected	Fastest sort